

Ho Chi Minh City University of Technology (HCMUT) — VNU-HCM

Faculty of Computer Science and Engineering

CO3133: Deep Learning and Its Applications

Assignment 02: Deep Learning on Large-Scale Data and Specialized Tasks

Text Classification Track — Six-Class Emotion Recognition on `dair-ai/emotion`: a TF-IDF Baseline versus Fine-Tuned DistilBERT and BERT

Final Report covering M1 (Proposal), M2 (Draft) and M3 (Final)

Course Code / Term:	CO3133 — Semester 261 (Academic Year 2026–2027)
Instructor:	Lê Thành Sách
Group:	Group Goat
Author & Lead:	**Võ Lê Thành Triệu** (Student ID **2453297**) — 100% of Assignment 02
Other Members:	Hoàng Trọng Huy Minh (2452743, 0% of A2; Assignment 1), Nguyễn Huỳnh Nam Quốc (2453089, 0% of A2; Assignment 3)
Dataset Proposal:	M1 Dataset Proposal 01 — status **Approved**
Repository:	`https://github.com/volethanhtrieu/DL_Assignment-02`
Assignment Page:	`https://volethanhtrieu.github.io/DL_Assignment-02/`
Experiment Tracking:	Weights & Biases project `goat-a2-emotion` (public, 12 runs)
Checkpoints:	Google Drive folder `DL` (view only): all 12 checkpoints in `A2/checkpoints/`; selected DistilBERT + selected baseline also as `A2/checkpoints_final.zip` (257.3 MB)
Model Hub & Demo:	Hugging Face model repo `TrieuVo/goat-a2-emotion` (selected model + baseline); live demo Space (both models run in the browser)
Slides:	`slides/Goat_A2_Slides.pdf` in the repository
Presentation Video:	YouTube: `youtube.com/watch?v=ok-hF_vicjc` (14:48)
Submission Date:	11 November 2026 (M3 Final, due 11 Nov 2026, 23:59 GMT+7)

Abstract

We classify short English texts into six emotions (sadness, joy, love, anger, fear, surprise) using the `split` configuration of `dair-ai/emotion`, pinned to one Hugging Face revision. A duplicate audit found 51 repeated-text groups with conflicting labels; after the approved filtering rules the splits contain 15,923 / 1,993 / 2,000 texts with no shared text between them. We compare a TF-IDF + logistic regression baseline (CPU) with full fine-tuning of DistilBERT and BERT (Colab T4), each transformer trained with three seeds. All choices (class weights, text cleaning, checkpoint epoch, final model) were made on validation Macro-F1 with rules fixed in advance; the test set was used once. On test, the selected model, DistilBERT full fine-tune (seed 42), reaches **accuracy 0.9270 and Macro-F1 0.8902** (three seeds: 0.8861 ± 0.0038), against 0.8905 / 0.8548 for the best baseline. The gain holds for all six classes and is largest on long texts, texts with "but" and negated texts. BERT is statistically tied with DistilBERT (0.8878 ± 0.0004; 62 vs. 60 texts that only one of the two gets right) at about twice the cost, and a frozen DistilBERT encoder fails under the same recipe (0.4210 ± 0.0020). The baseline is about 45× faster per text on the same CPU. Error analysis shows that most remaining errors sit between close emotions (joy/love, the negative emotions, fear/surprise) and that part of them point to doubtful labels.

Contents

I Problem and Data **4**

1 Problem Formulation **4**
1.1 Motivation . . . 4
1.2 Input, Output and Formulation . . . 4
1.3 Main Difficulties . . . 4

2 Dataset Description **5**

3 Exploratory Data Analysis **5**
3.1 Class Distribution and Imbalance . . . 5
3.2 Text Length . . . 5
3.3 Duplicates and Conflicting Labels . . . 6
3.4 Vocabulary and Class-Specific Words . . . 6

4 Data Splitting and Leakage Prevention **7**

5 Preprocessing **8**

II Methodology **8**

6 Pipeline Overview **8**

7 Models **9**
7.1 Baseline: TF-IDF + Logistic Regression . . . 9
7.2 Main Model: DistilBERT, Full Fine-Tuning . . . 11
7.3 Second Pretrained Model: BERT, Full Fine-Tuning . . . 11
7.4 Controlled Variant: Frozen DistilBERT Encoder . . . 12
7.5 Self-Implemented Parts and Libraries . . . 12

8 Training Setup **12**

9 Experimental Design **13**

III Results **14**

10 Training Behaviour **14**

11 Quantitative Results **16**
11.1 Validation (used for every decision) 16
11.2 Test (evaluated once, after selection) 16

12 Controlled Experiments and Ablation **17**

13 Compute Cost **18**

14 Qualitative Results **18**

15 Error Analysis **19**
15.1 Confusion Pairs 20
15.2 Error Groups 20
15.3 Label Noise and Text Properties 21

16 Comparison and Discussion **22**

17 Limitations and Conclusion **23**

18 Reproducibility **23**

19 AI Usage Disclosure **24**
19.1 Declaration 25

Part I
Problem and Data

1 Problem Formulation

1.1 Motivation

Recognising the emotion expressed in a short text is useful for customer-feedback triage, social-media monitoring and mental-health support tools, where people describe how they feel in one or two sentences. The task is harder than sentiment polarity: several emotions share vocabulary ("I feel so lucky to have him" can be read as joy or love), a single text can contain more than one emotion, and the labels of public datasets are noisy.

1.2 Input, Output and Formulation

The **input** is one English text x (a sequence of words); the **output** is one label $y \in \mathcal{Y} = \{0{:}\text{sadness}, 1{:}\text{joy}, 2{:}\text{love}, 3{:}\text{anger}, 4{:}\text{fear}, 5{:}\text{surprise}\}$. The prediction unit is one text record. Given a training set $\mathcal{D} = \{(x_i, y_i)\}_{i=1}^{N}$, a model f_θ outputs logits $\mathbf{z} = f_\theta(x) \in \mathbb{R}^6$ and class probabilities

$$p_\theta(y = k \mid x) = \frac{\exp(z_k)}{\sum_{j=0}^{5} \exp(z_j)}, \qquad \hat{y} = \arg\max_k p_\theta(y = k \mid x). \tag{1}$$

Parameters are learned by minimising a class-weighted cross-entropy (Section 8). Because the classes are imbalanced, the primary metric is Macro-F1, the unweighted mean of the per-class F1 scores, with accuracy reported alongside it (handbook §20.1):

$$\text{Macro-F1} = \frac{1}{6}\sum_{c=0}^{5} \frac{2\,P_c R_c}{P_c + R_c}, \qquad \text{Accuracy} = \frac{1}{N}\sum_{i=1}^{N} \mathbb{1}[\hat{y}_i = y_i]. \tag{2}$$

Accuracy alone would hide failures on the small classes: always predicting *joy* already gives 0.3475 accuracy on test but only 0.0860 Macro-F1.

1.3 Main Difficulties

- **Class imbalance:** joy has 9.37 times as many training texts as surprise.
- **Label noise:** identical texts appear with different labels (Section 3.3); the dataset card describes the annotations as machine-generated.
- **Close classes and negation:** texts are mostly of the form "i feel …" followed by an emotion word, so the label often depends on one word, which can be negated or contrasted ("…but …").

2 Dataset Description

Table 1: Dataset specification.

Property	Value
Name and source	Emotion, `dair-ai/emotion` on the Hugging Face Hub, configuration `split` [1]
Version	Revision `cab853a1dbdf4c42c2b3ef2173804746df8825fe` (pinned in the notebook)
License	Listed as "other"; the dataset card and the authors' repository restrict use to educational and research purposes
Labelling	One categorical label per text; the card lists the annotations as machine-generated and the collection process is only partly described
Size	20,000 texts in the official splits: 16,000 train / 2,000 validation / 2,000 test (80/10/10%)
Format	Two fields, `text` (string) and `label` (integer 0–5); loaded from the Parquet files of the pinned revision; storage size 1.29 MB as three Parquet files (train 1.03 MB, validation 0.13 MB, test 0.13 MB; 1,287,193 bytes in total) and 2.17 MB once loaded (dataset card of the pinned revision)
Validity	No missing, blank or non-string text and no invalid label in any split
Known biases	English only; informal first-person texts dominated by "i feel …"; imbalanced classes; conflicting duplicates (below)

3 Exploratory Data Analysis

3.1 Class Distribution and Imbalance

Figure 1 shows the label distribution per split. All splits contain all six classes in similar proportions. In the official training split joy is the largest class (5,362 texts, 33.5%) and surprise the smallest (572, 3.6%), an imbalance ratio of 9.37. Two consequences guided the design: Macro-F1 is used for every decision, and class weights were tested as an explicit ablation rather than adopted by default (Section 9).

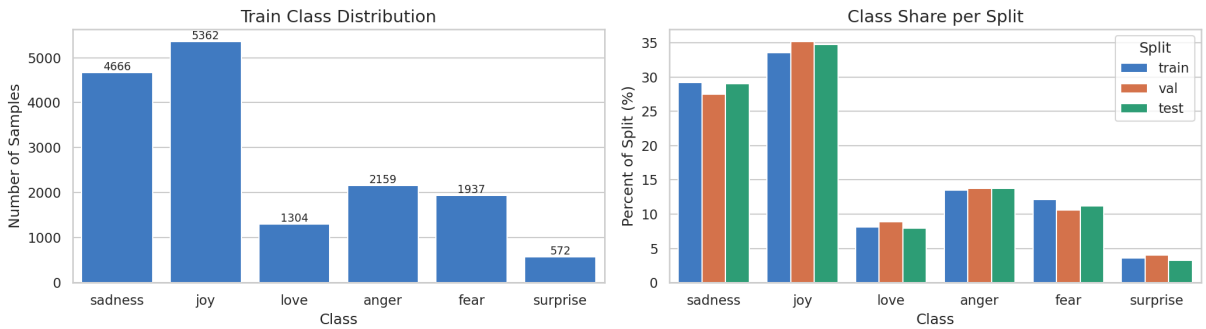


Figure 1: Class distribution per split (official release, before filtering).

3.2 Text Length

Texts are short. On the official training split the median length is 86 characters / 17 words (mean 96.85 / 19.17, 95th percentile 207 / 41, maximum 300 / 66); the longest validation and test texts

have 61 words. Per-class median lengths lie between 16 and 19 words, so length alone does not reveal the class. With the DistilBERT WordPiece tokenizer (Section 5) a text has a median of 20 tokens including `[CLS]` and `[SEP]`, a 95th percentile of 45, a 99th percentile of 57 and a maximum of 87 (train), 69 (val) and 66 (test); there are 1.06 tokens per word. No text exceeds 128 tokens, so the planned maximum length of 128 never truncates.

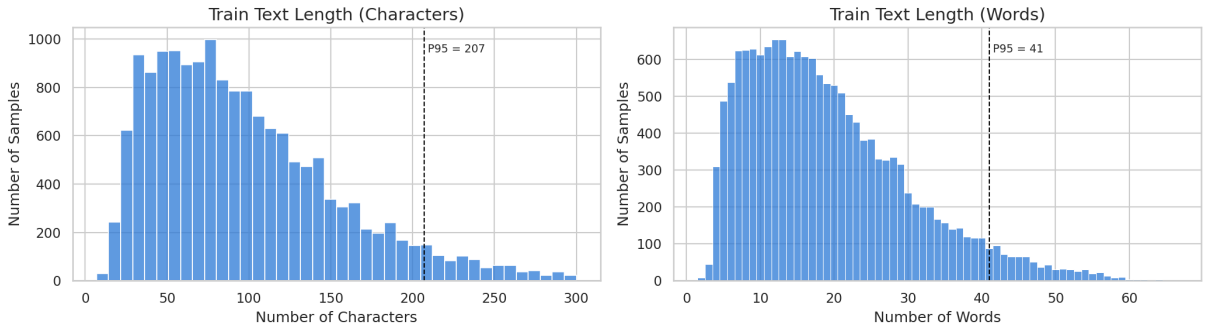


Figure 2: Distribution of text length per split.

3.3 Duplicates and Conflicting Labels

Texts were compared after a normalisation of Unicode NFC, lowercasing, trimming and collapsing whitespace; exact and normalised comparison gave the same groups (Table 2). Over all 20,000 texts there are 52 repeated-text groups, and **51 of them carry conflicting labels** (102 rows, two per group). Fourteen of the conflicting groups include a test text. These rows show two problems: *leakage* (the same text in two splits) and *label noise* (the same text with two emotions).

Table 2: Repeated or shared text groups (official release).

Scope	Groups	Scope	Groups
Within train	31	Train–validation overlap	5
Within validation	2	Train–test overlap	11
Within test	0	Validation–test overlap	3

3.4 Vocabulary and Class-Specific Words

The training vocabulary (minimal cleaning) has 15,188 word types; 51.5% of them occur only once, and 2.72% / 2.74% of the validation / test tokens are not in the training vocabulary. “feel” occurs in 66.4% of training texts and “feeling” in 31.1%. The most class-specific n-grams are the emotion adjectives themselves (Figure 3; e.g. *offended*, *intimidated*, *amazed*), many of which appear in a single class only. This suggested that the labels largely follow one emotion word, a hypothesis that the error analysis later tests.

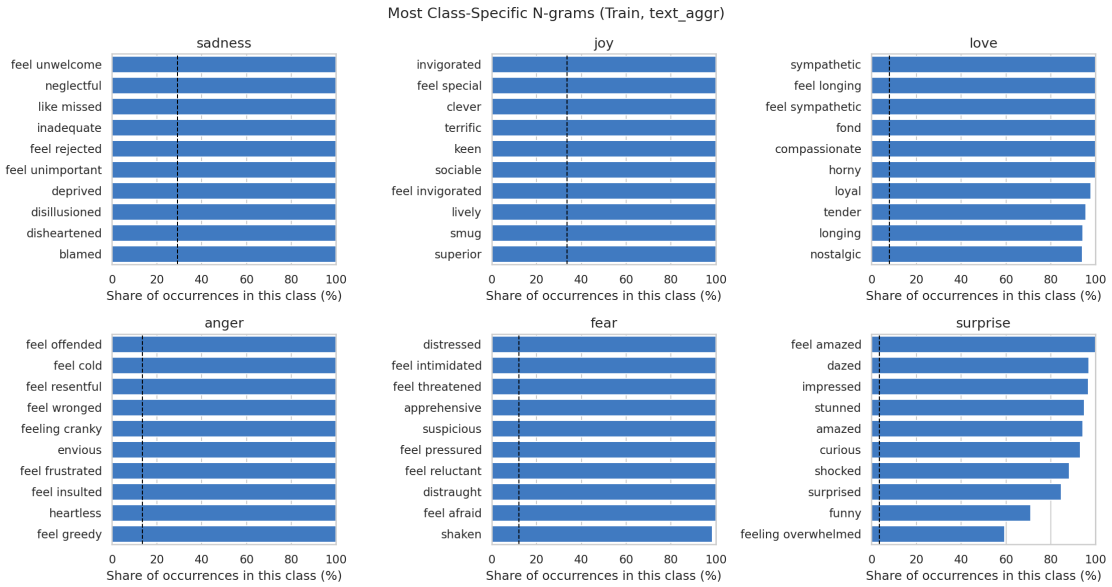


Figure 3: Most frequent and most class-specific n-grams per class (training split, stopwords removed).

4 Data Splitting and Leakage Prevention

The official train/validation/test splits are kept (no re-splitting, no stratification needed, seed 42 for everything random). The split unit is a *group of texts with the same normalised text*: such a group may not cross splits. The four filtering rules approved in the proposal were applied in order:

1. keep the test set unchanged, including its released labels, and remove every training or validation text that also occurs in test;
2. for remaining train–validation overlaps, keep the validation copy and remove the training copy;
3. inside each of train and validation, remove every group with conflicting labels (no label is chosen or invented);
4. for same-label duplicates, keep the first occurrence.

Table 3: Rows removed by rule and final split sizes.

Reason	Train	Validation	Test
Text also in test (rule 1)	11	3	0
Text also in validation (rule 2)	5	–	–
Conflicting-label group (rule 3)	60 (30 groups)	4 (2 groups)	0
Same-label duplicate (rule 4)	1	0	0
Removed	**77**	**7**	**0**
Final size	**15,923 (79.95%)**	**1,993 (10.01%)**	**2,000 (10.04%)**

Seven checks pass after filtering: no shared normalised text between any two splits, no repeated text inside a split, at least 5,000 training texts and all six classes in every split. The class proportions stay close to the original (train: joy 33.54%, sadness 29.27%, anger 13.52%, fear 12.08%, love 8.06%, surprise 3.54%). Everything learned from data (TF-IDF vocabulary and

IDF, class weights) is fitted on the training split only. One limitation remains: the five train–validation overlaps all had conflicting labels, so the kept validation copies and the 14 test texts that had a copy elsewhere carry disputed labels.

5 Preprocessing

- **Minimal text** (used by all transformers and two baselines): Unicode NFC, lowercase, single spaces. Stopwords are kept because transformers use them as context. The released texts contain no apostrophes, so "I'm" appears as *im* and "don't" as *dont* or *don t*.
- **Aggressive text** (one baseline variant): minimal text, then only letters, digits and spaces are kept and the 318 scikit-learn English stopwords are removed *except* 23 negation and intensity words (not, no, never, nothing, without, very, so, too, more, most, but, …). The average training text drops from 19.18 to 9.31 tokens, and no text becomes empty.
- **Tokenisation for transformers:** the model's own uncased WordPiece tokenizer adds `[CLS]` and `[SEP]`; truncation at 128 tokens (never reached); padding *per batch* to the longest text of the batch. Padded positions get attention mask 0, so they do not change the predictions. Per-batch padding needs only 36.1% of the tokens that padding everything to 128 would need on the validation set. BERT and DistilBERT produce identical token ids on all three splits, so both use the same data loaders.
- **Labels:** kept as integers 0–5 with the mapping above. No augmentation is used.

One example batch after preprocessing. The first validation batch has the shapes below (batch size 16; the longest text in it has 34 tokens, so 52.8% of the batch is padding):

Table 4: Tensors of one validation batch and one decoded text from it.

Tensor	Shape / type	Content
input ids	$(16, 34)$, int64	WordPiece ids, one row per text
attention mask	$(16, 34)$, int64	1 for real tokens, 0 for padding
labels	$(16,)$, int64	class ids, equal to the validation labels
model output	$(16, 6)$	one logit per class
Example text	"im feeling quite sad and sorry for myself but ill snap out of it soon" (label 0, sadness)	
Tokens	[CLS] im feeling quite sad … soon [SEP] followed by $17 \times$ [PAD]	

Part II
Methodology

6 Pipeline Overview

Figure 4 shows the end-to-end pipeline. The two model families share the data, the splits, the filtering and the evaluation; they differ in representation and training.

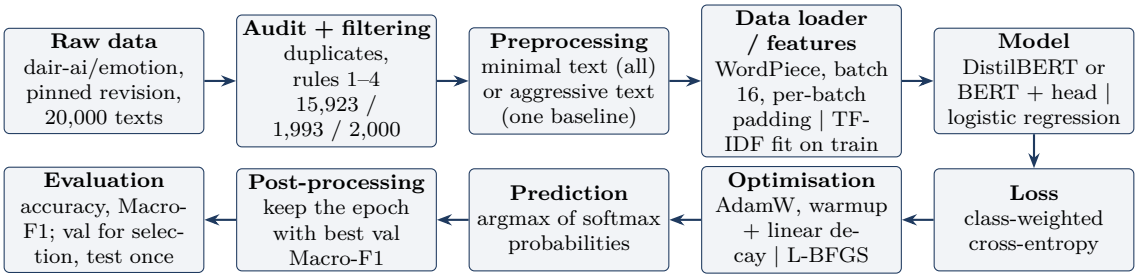


Figure 4: End-to-end pipeline in the form required by the handbook (read the second row from right to left). Where a box has two parts, the left part is the transformer path and the part after the bar is the baseline path. The only post-processing is keeping the best epoch of each transformer run.

7 Models

7.1 Baseline: TF-IDF + Logistic Regression

The baseline is a classical bag-of-n-grams classifier [2, 9]. It has two stages, a TF-IDF feature extractor and a logistic-regression classifier. Both are *fitted on the training split only* and then applied unchanged to validation and test, so no information from those splits enters the features (Figure 5).

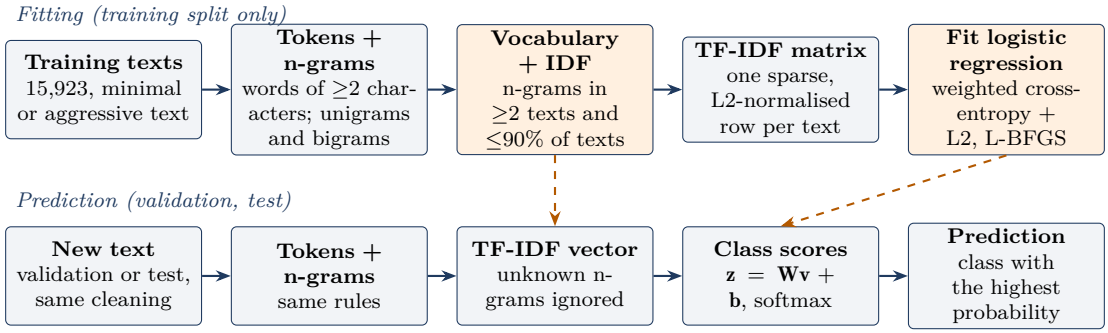


Figure 5: TF-IDF + logistic-regression pipeline. Orange boxes are the only learned parts; dashed arrows show that validation and test texts reuse the vocabulary, IDF weights and classifier learned from the training split.

Stage 1: TF-IDF features.

1. *Tokens and n-grams.* The cleaned text is split into words of at least two letters or digits (the library default), so the pronoun "i", present in 93.2% of training texts, is not a feature. Every word (unigram) and every pair of adjacent words (bigram) is a candidate feature; bigrams keep a little word order, e.g. "not happy" differs from "happy".
2. *Vocabulary.* Only n-grams that occur in at least 2 training texts (this removes typos and one-off phrases) and in at most 90% of them (this removes near-universal terms) are kept: 32,776 n-grams with minimal text and 17,070 with aggressive text. Each kept n-gram is one dimension of the feature vector.
3. *Weighting.* The weight of n-gram t in text d is

$$\mathrm{tfidf}(t,d) = \mathrm{tf}(t,d) \cdot \left(\ln \frac{1+n}{1+\mathrm{df}(t)} + 1 \right), \tag{3}$$

where $\mathrm{tf}(t,d)$ is the count of t in d, n the number of training texts and $\mathrm{df}(t)$ the number of training texts that contain t. An n-gram that is frequent in the text but rare in the corpus (an emotion adjective) gets a large weight; a common one ("feel", in 66.4% of texts) a small one.

4. *Normalisation.* Each vector is divided by its L2 norm, so long and short texts are on the same scale. Almost all entries are zero, because a text contains only a few of the thousands of n-grams.

For example, the training text "i do not feel reassured anxiety is on each side" (label joy) becomes "not feel reassured anxiety" after aggressive cleaning; its candidate features are the unigrams *not, feel, reassured, anxiety* and the bigrams *not feel, feel reassured, reassured anxiety.* Those in the vocabulary receive a TF-IDF weight; every other dimension is zero.

Stage 2: logistic regression. A multinomial logistic regression maps the vector $\mathbf{v} \in \mathbb{R}^F$ (F = vocabulary size) to six class scores and probabilities,

$$\mathbf{z} = \mathbf{W}\mathbf{v} + \mathbf{b}, \qquad p(y = k \mid \mathbf{v}) = \frac{\exp z_k}{\sum_{j=1}^{6} \exp z_j}, \qquad \mathbf{W} \in \mathbb{R}^{6 \times F}, \tag{4}$$

so each weight $W_{k,t}$ says how strongly n-gram t pushes a text towards class k. Training minimises the class-weighted cross-entropy plus an L2 penalty,

$$\min_{\mathbf{W},\mathbf{b}} \ \tfrac{1}{2}\|\mathbf{W}\|^2 + C \sum_{i=1}^{N} w_{y_i}\big(-\log p(y_i \mid \mathbf{v}_i)\big), \qquad w_c = \frac{N}{6N_c}, \quad C = 1, \tag{5}$$

with the L-BFGS optimiser (at most 1,000 iterations). The balanced weights w_c make each class contribute equally to the loss although, after filtering, joy has 5,340 training texts and surprise only 564. The prediction is the class with the highest probability. The model has $6 \times (F + 1)$ parameters: 196,662 with minimal text and 102,426 with aggressive text.

Variants and role. Two text inputs are compared (Section 5): minimal text, and aggressive text that also removes 295 stopwords while keeping negations and intensifiers. Class weights on vs. off is a further ablation. The selected baseline (aggressive text, class weights) was chosen on validation (experiment E3). The baseline answers one question: how far do word and word-pair cues alone go on this dataset? *Strengths:* trains in seconds on one CPU core and is fully interpretable, since the largest weights of each class can be read directly (Figure 6). *Limits:* no word order beyond bigrams, no notion of negation scope, and no knowledge of words unseen in training.

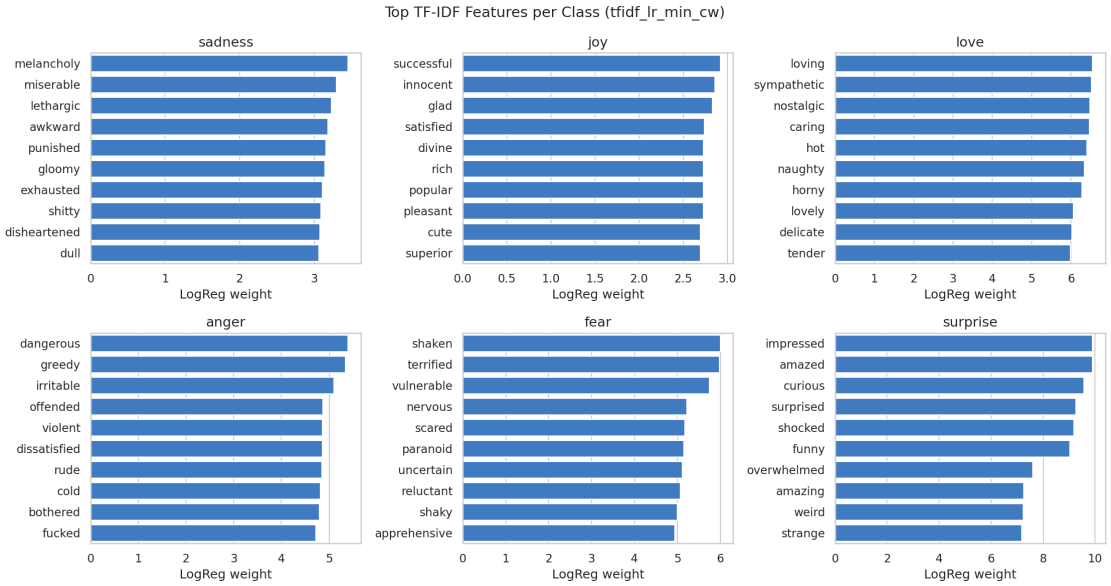


Figure 6: Largest logistic-regression weights per class (baseline with class weights, minimal text). The top features are single emotion adjectives.

7.2 Main Model: DistilBERT, Full Fine-Tuning

DistilBERT (`distilbert-base-uncased`) [5] is a 6-layer Transformer encoder (hidden size 768, 12 attention heads) distilled from BERT. The final hidden state of `[CLS]`, $\mathbf{h} \in \mathbb{R}^{768}$, goes through the standard sequence-classification head:

$$\mathbf{z} = \mathbf{W}_2\,\mathrm{Dropout}\big(\mathrm{ReLU}(\mathbf{W}_1\mathbf{h} + \mathbf{b}_1)\big) + \mathbf{b}_2, \qquad \mathbf{W}_1 \in \mathbb{R}^{768\times768},\ \mathbf{W}_2 \in \mathbb{R}^{6\times768}. \tag{6}$$

The encoder has 66,362,880 parameters and the new head 595,206 (590,592 + 4,614), 66,958,086 in total. In full fine-tuning every parameter is updated. *Strengths:* contextual representations that can model negation and contrast. *Limits:* about 45× slower than the baseline per text on CPU and needs a GPU to train in minutes.

7.3 Second Pretrained Model: BERT, Full Fine-Tuning

BERT (`bert-base-uncased`) [4] has 12 layers of the same width. Its classification head is a single linear layer (4,614 parameters) on top of the *pretrained* pooler (a dense layer with tanh on `[CLS]`), so 109,482,240 of its 109,486,854 parameters are pretrained. BERT was added after the proposal (plan update of 27 Oct 2026, recorded in the repository README) because time remained: it uses exactly the same recipe as DistilBERT, to see what a larger model changes in quality and cost.

Figure 7 compares the two architectures. Both read the same WordPiece token ids (checked to be identical on every split) and differ in depth and in the classification head.

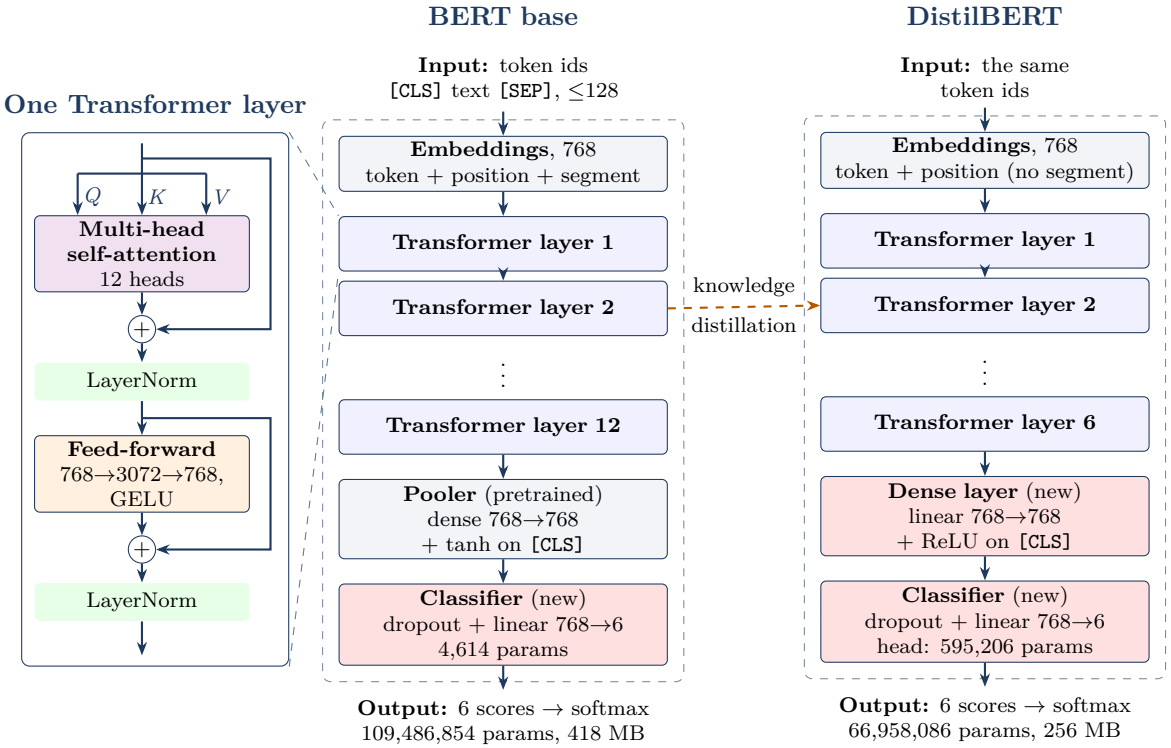


Figure 7: BERT base and DistilBERT as used here. Left: one Transformer layer [3]; the input of each sub-block is added back to its output (residual connection) before LayerNorm, and every token leaves the layer as a 768-dimensional vector that depends on the whole sentence. Grey and blue boxes are pretrained and are fine-tuned (frozen in the frozen-encoder variant); red boxes are new and start from random weights. DistilBERT keeps BERT's width but half the layers, and was trained to imitate BERT (distillation) [5].

7.4 Controlled Variant: Frozen DistilBERT Encoder

The same DistilBERT model with the whole encoder frozen: only the head (595,206 parameters, 0.89% of the model) is trained. This is the changed factor of the main controlled experiment.

Table 5: Models compared. Trainable parameters are those updated by the optimiser.

Model	Input representation	Total params	Trainable	Size (MB)
TF-IDF + LR (minimal text)	TF-IDF, 1–2-grams	196,662	196,662	2.23
TF-IDF + LR (aggressive text)	TF-IDF, 1–2-grams	102,426	102,426	1.17
DistilBERT, full fine-tune	WordPiece ids, ≤128 tokens	66,958,086	66,958,086	256.12
DistilBERT, frozen encoder	WordPiece ids, ≤128 tokens	66,958,086	595,206	256.12
BERT, full fine-tune	WordPiece ids, ≤128 tokens	109,486,854	109,486,854	418.36

7.5 Self-Implemented Parts and Libraries

Libraries: Hugging Face *datasets* (loading at the pinned revision) and *transformers* [7] (pre-trained models, tokenizers, per-batch padding collator, linear warmup scheduler); PyTorch [8] (AdamW, cross-entropy, data loaders); scikit-learn [9] (TF-IDF, logistic regression, class-weight computation, metrics); Weights & Biases [11] (experiment logging). **Written by the group:** the duplicate audit and the four filtering rules with their checks, both text-cleaning variants, the training and evaluation loop (two learning rates, best-epoch checkpointing, history files, resume after a Colab disconnect), the encoder-freezing function, the selection rules, the repeated timing protocol and the error-analysis grouping.

8 Training Setup

The transformer loss is the class-weighted cross-entropy (PyTorch mean reduction):

$$\mathcal{L}(\theta) = -\frac{\sum_{i \in B} w_{y_i} \log p_\theta(y_i \mid x_i)}{\sum_{i \in B} w_{y_i}}, \qquad w_c = \frac{N}{6\, N_c}, \tag{7}$$

with weights computed from the filtered training labels: sadness 0.5694, joy 0.4970, love 2.0685, anger 1.2332, fear 1.3800, surprise 4.7054. The validation loss uses the same weights. Both learning rates follow linear warmup over the first 10% of the $T = 2{,}988$ optimiser steps (996 batches × 3 epochs), then linear decay to zero:

$$\eta_t = \eta_{\max} \cdot \begin{cases} t/T_w, & t < T_w = 298, \\ (T - t)/(T - T_w), & t \geq T_w. \end{cases} \tag{8}$$

Table 6: Training configuration (identical for every transformer run).

Setting	Value
Optimiser	AdamW [6], weight decay 0.01
Learning rate	encoder 2×10^{-5}, head 1×10^{-4} (two parameter groups)
Scheduler	linear warmup (10% of steps, 298) then linear decay to 0
Batch size / epochs	16 / 3 (2,988 optimiser steps); training batches shuffled with a seeded generator, the same order for every model with the same seed
Regularisation	weight decay, gradient-norm clipping at 1.0, dropout of the pretrained models
Early stopping	none: all 3 epochs are run
Checkpoint criterion	the epoch with the highest validation Macro-F1 is saved and reloaded
Seeds	42, 43, 44 (seed 42 = main run of each model)
Precision	FP32 (no mixed precision)
Hardware	Google Colab, NVIDIA Tesla T4 (14.6 GB), Intel Xeon 2.00 GHz × 2 vCPU, 12.7 GB RAM; baselines on CPU
Software	Python 3.13.16, PyTorch 2.14.1 (CUDA 13.0), transformers 5.22.1, scikit-learn 1.6.1, datasets 4.8.5, wandb 0.28.1
Training time	DistilBERT 276.9 ± 10.5 s, BERT 524.7 ± 0.5 s, frozen DistilBERT 81.4 ± 0.4 s per run (T4); baselines 5.3–20.7 s (CPU)
Hyperparameter choice	no search: initial values common for BERT-style fine-tuning; only class weights and the baseline text variant were chosen, on validation

A sanity check before training gave a starting loss of 1.7926 for a random head, close to $\ln 6 = 1.7918$, as expected for six classes.

9 Experimental Design

Every comparison changes one factor and was decided on **validation** with a rule written down before the runs. Table 7 lists the four experiments.

Table 7: Experiments: hypothesis, changed and fixed factors, metric and decision criterion.

Experiment	Hypothesis	Changed / fixed	Metric	Decision criterion
E1 (main): frozen encoder vs. full fine-tune (DistilBERT)	The pretrained `[CLS]` features are not tuned to emotions, so training only the head gives clearly lower Macro-F1, but trains faster.	Changed: encoder frozen or not. Fixed: data, text, tokens, batch order, seeds, head LR, schedule, class weights, 3 epochs.	Val Macro-F1, mean ± std over 3 seeds; accuracy, per-class F1; cost	(1) Real gap if full is higher on all 3 seeds and the mean gap > 2× the larger seed std. (2) Frozen is "good enough" if its mean drop ≤ 0.02.
E2: BERT vs. DistilBERT (both full)	The deeper model may give a small gain at about twice the cost; the question is whether the gain exceeds seed noise.	Changed: pretrained model. Fixed: everything else, including identical token ids and batch order.	Val Macro-F1, mean ± std and per seed	|mean gap| < larger seed std ⇒ tie ⇒ keep the smaller model.
E3 (baseline): stopwords removed vs. kept	The signal sits in emotion adjectives, which both texts keep, so Macro-F1 changes little.	Changed: text column. Fixed: TF-IDF settings, C, class weights.	Val Macro-F1	Higher wins; a tie keeps the minimal text.
E4 (ablation): class weights on vs. off (baseline)	Balanced weights raise recall of the small classes (surprise, love) at some cost in precision, so Macro-F1 rises more than accuracy.	Changed: class weights. Fixed: minimal text, TF-IDF settings, $C = 1$.	Val Macro-F1; per-class P/R/F1	Higher wins (a tie keeps no weights). The winner is used by every later run.

Model selection (fixed before any test result). The baseline is the better text variant of E3. The main model is chosen by E2's rule, and its seed-42 run is used, not the best seed, to avoid one more selection on validation. The frozen encoder is excluded if it fails criterion (2) of E1. Only after these choices were the 12 runs evaluated once on test; test results changed nothing. Differences between two models on the same texts are also checked with McNemar's test [10] on the texts that exactly one model gets right.

Part III
Results

10 Training Behaviour

Figure 8 shows the seed-42 runs. The full fine-tuning runs converge within the 3 epochs: training loss falls from 0.67 (DistilBERT) and 0.80 (BERT) at epoch 1 to about 0.12, and validation Macro-F1 is above 0.89 after the first epoch. The validation loss is flat at epochs 2–3 (0.20 for both models) and rose slightly at epoch 3 for three of the six full runs while Macro-F1 still rose, so the runs are at the edge of overfitting but not past it; the 3-epoch budget is adequate. The frozen-encoder runs *underfit*: training and validation loss are equal (1.425 at epoch 3) and still falling when the learning rate reaches zero.

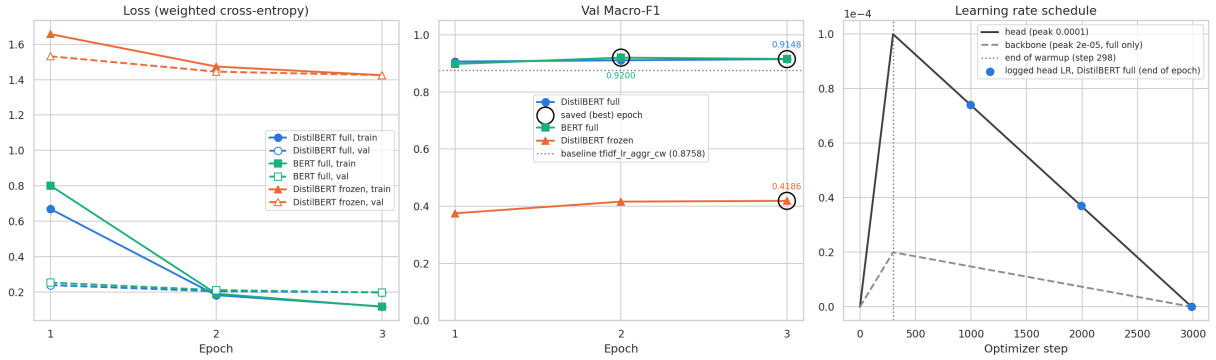


Figure 8: Training curves of the seed-42 runs: weighted cross-entropy (left), validation Macro-F1 with the saved epoch circled and the selected baseline as a dotted line (middle), learning rate per step (right). Colour = run (blue DistilBERT, green BERT, orange frozen DistilBERT); in the loss panel solid = training, dashed = validation.

Table 8: Selected checkpoint (best validation Macro-F1 epoch) of every transformer run.

Model	Seed 42	Seed 43	Seed 44
DistilBERT, full fine-tune	epoch 3 (0.9148)	epoch 3 (0.9178)	epoch 3 (0.9140)
BERT, full fine-tune	epoch 2 (0.9200)	epoch 3 (0.9145)	epoch 3 (0.9145)
DistilBERT, frozen encoder	epoch 3 (0.4186)	epoch 2 (0.4132)	epoch 3 (0.4253)

Implementation issues and fixes.

- The Assignment 1 training function left the first linear layer of the DistilBERT head out of the optimiser; the head is now defined as every parameter outside the encoder, so both head layers are trained.
- A first run stopped with a name error because the import cell had not been re-run after adding the padding collator; re-running it fixed the problem.
- Single inference timings varied by 10–20% between runs; timings are now repeated five times after a warm-up and the median is reported (Section 13).
- Test metrics are computed after training, so each run's W&B identifier is stored and the run is resumed to add its test scores.
- The whole notebook was rerun from scratch on 8 Nov 2026 with all checkpoints deleted: every validation metric, history and per-class file was identical to the earlier runs; only timings changed.

11 Quantitative Results

11.1 Validation (used for every decision)

Table 9: Validation results (1,993 texts). Transformers: mean ± std over seeds 42/43/44.

Model	Setup	Runs	Accuracy	Macro-F1
Majority class (joy)	–	–	0.3522	0.0868
TF-IDF + LR	minimal text, no class weights	1	0.8289	0.7527
TF-IDF + LR	minimal text, class weights	1	0.8751	0.8489
TF-IDF + LR	aggressive text, class weights	1	0.8991	0.8758
DistilBERT	frozen encoder	3	0.4874 ± 0.0033	0.4191 ± 0.0061
DistilBERT	full fine-tune	3	0.9373 ± 0.0005	0.9155 ± 0.0020
BERT	full fine-tune	3	0.9383 ± 0.0031	0.9163 ± 0.0032

Applying the rules: the aggressive-text baseline (`tfidf_lr_aggr_cw`) is the selected baseline (E3); BERT minus DistilBERT is +0.0008, smaller than the larger seed std 0.0032, so the two are tied and the smaller model, **DistilBERT full fine-tune, seed 42** (`distil_full_min`), is the final model (E2).

11.2 Test (evaluated once, after selection)

Table 10: Test results (2,000 texts) of all 12 runs.

Model	Setup	Seed	Val F1	Test Acc	Test F1	Test−Val
DistilBERT	**full (selected)**	**42**	**0.9148**	**0.9270**	**0.8902**	−0.0246
DistilBERT	full	43	0.9178	0.9230	0.8854	−0.0324
DistilBERT	full	44	0.9140	0.9245	0.8828	−0.0313
BERT	full	42	0.9200	0.9280	0.8881	−0.0318
BERT	full	43	0.9145	0.9245	0.8877	−0.0268
BERT	full	44	0.9145	0.9270	0.8874	−0.0271
TF-IDF + LR	aggr., weights (selected)	–	0.8758	0.8905	0.8548	−0.0210
TF-IDF + LR	minimal, weights	–	0.8489	0.8695	0.8365	−0.0124
TF-IDF + LR	minimal, no weights	–	0.7527	0.8325	0.7289	−0.0239
DistilBERT	frozen	42	0.4186	0.5010	0.4199	+0.0013
DistilBERT	frozen	43	0.4132	0.4940	0.4199	+0.0066
DistilBERT	frozen	44	0.4253	0.4970	0.4233	−0.0020
Majority class (joy)	–	–	0.0868	0.3475	0.0860	–

Over seeds, test Macro-F1 is 0.8861 ± 0.0038 for DistilBERT, 0.8878 ± 0.0004 for BERT and 0.4210 ± 0.0020 for the frozen encoder. The selected model gets 1,854 of 2,000 test texts right (146 errors), the selected baseline 1,781 (219 errors). The ranking of all setups is the same on validation and test. The strong models lose 0.02–0.03 Macro-F1 from validation to test; Table 11 shows that this drop comes mostly from surprise (DistilBERT −0.082 F1, only 66 test texts) and love (−0.033), while sadness and fear barely change. Part of the drop is expected, because the checkpoint epoch and the model were chosen on validation.

Table 11: Per-class F1 on test (main runs, seed 42) and validation F1 of the selected model.

Class	Test texts	Baseline	DistilBERT	BERT	Frozen	DistilBERT val
sadness	581	0.9296	0.9612	0.9643	0.5299	0.9616
joy	695	0.9110	0.9504	0.9513	0.6492	0.9561
love	159	0.7925	0.8523	0.8452	0.3435	0.8853
anger	275	0.8917	0.9134	0.9202	0.4329	0.9375
fear	224	0.8631	0.8904	0.8825	0.4219	0.8929
surprise	66	0.7407	0.7737	0.7654	0.1421	0.8554
Macro-F1	2,000	0.8548	0.8902	0.8881	0.4199	0.9148

12 Controlled Experiments and Ablation

Table 12 gives each comparison as A − B in Macro-F1, on validation (where it was decided) and on test (check only).

Table 12: Experiment results (A − B, Macro-F1). "A higher" counts seeds or runs.

Comparison (A − B)	Val mean	Val: A higher	Test mean	Test: A higher
E1: full − frozen (DistilBERT)	+0.4965	3 of 3	+0.4651	3 of 3
E2: BERT − DistilBERT (full)	+0.0008	2 of 3	+0.0016	2 of 3
E3: aggressive − minimal text	+0.0269	1 of 1	+0.0183	1 of 1
E4: class weights on − off	+0.0961	1 of 1	+0.1076	1 of 1

E1, frozen vs. full (main). Full fine-tuning is higher on all three seeds and the mean gap (0.4965) is far above twice the larger seed std (0.0122): criterion (1) holds. The frozen encoder loses 0.50 Macro-F1, far more than the allowed 0.02, and even scores below the baseline: criterion (2) fails, so it was excluded. The hypothesis is supported. The frozen head has errors spread over all classes (it predicts fear 409 times for 211 true fear texts on validation) and is underfitted (Section 10). It trains in 29% of the time of full fine-tuning but is *not* faster at inference, since the forward pass is the same. This result holds for this recipe only: the head kept the learning rate and the 3 epochs of full fine-tuning.

E2, BERT vs. DistilBERT. Per seed, BERT − DistilBERT is +0.0051, −0.0033, +0.0005 on validation and −0.0021, +0.0023, +0.0047 on test. Both mean gaps are below the seed std, so the two models are tied; on test, 62 texts are right only for BERT and 60 only for DistilBERT (McNemar exact $p \approx 0.93$). The hypothesis of a small gain is not supported at the precision three seeds allow. The changed factor bundles depth, the pretrained pooler and distillation, so the experiment says *whether* BERT differs, not *why*.

E3, stopwords (baseline). Removing stopwords with the negation keep-list *improved* the baseline on validation (+0.0269; F1 higher for all six classes; 85 texts right only with aggressive text vs. 37 only with minimal text, McNemar $p \approx 2 \times 10^{-5}$) and on test (+0.0183), with half as many n-grams. The hypothesis "little change" is therefore rejected. On validation texts with a negation word the accuracy rose from 0.8462 to 0.8910. A likely reason is that removing function words places the negation next to the emotion word, which creates useful bigrams such as "not happy". Caveats: one run per setting, untuned TF-IDF and C, and the aggressive text also removes non-alphanumeric characters.

E4, class weights (ablation). Balanced weights raise validation Macro-F1 from 0.7527 to 0.8489 and test Macro-F1 from 0.7289 to 0.8365, much more than accuracy (+0.046 val). As predicted, the small classes gain recall and lose precision (surprise recall 0.40 → 0.81, precision 0.89 → 0.73 on validation), and F1 improves for all six classes. Weights were therefore used for every later run, transformers included.

13 Compute Cost

Inference was timed with one warm-up and five repeats, keeping the median. Two settings are reported: the whole validation set in batches (baseline on CPU, transformers on the T4 with batches of 16, tokenisation included), and one text at a time for 100 validation texts, which is the only like-for-like comparison of all models on the same CPU (one PyTorch thread).

Table 13: Cost of the main runs. Training time: mean ± std over seeds; test Macro-F1: mean over seeds.

				Batched	One text (ms)		
Model	**Trainable**	**MB**	**Train (s)**	**ms/text**	**CPU**	**GPU**	**Test F1**
TF-IDF + LR (aggr.)	102,426	1.17	5.3 (CPU)	0.0148 (CPU)	1.32	–	0.8548
DistilBERT full	66,958,086	256.12	276.9 ± 10.5	1.63 (T4)	59.64	5.92	0.8861
DistilBERT frozen	595,206	256.12	81.4 ± 0.4	1.59 (T4)	60.90	4.71	0.4210
BERT full	109,486,854	418.36	524.7 ± 0.5	3.16 (T4)	119.19	8.79	0.8878

Relative to DistilBERT, BERT costs 1.64× the parameters, 1.89× the training time and 2.00× the CPU latency for +0.0016 test Macro-F1, a difference within seed noise. The baseline is about 45× faster than DistilBERT per text on the same CPU and 219× smaller on disk, for −0.0314 mean test Macro-F1 (−0.0354 against the selected seed-42 run). GPU memory was not measured, because several models shared the GPU during the run.

14 Qualitative Results

Table 14 shows test texts that all models get right, difficult texts that only the fine-tuned transformer gets right, and failures. The difficult cases were found by filtering the test predictions for texts with “but” or a negation that DistilBERT gets right and the baseline does not; the failures are the random examples (seed 42) printed by the error-analysis cells.

Table 14: Test texts with the true label and the predictions of the three main models (D = DistilBERT, B = baseline, T = BERT; ✓ = correct).

Case	Text	True	D	B	T
Easy	im updating my blog because i feel shitty	sadness	✓	✓	✓
Easy	i felt anger when at the end of a telephone call	anger	✓	✓	✓
Easy	i find myself in the odd position of feeling supportive of	love	✓	✓	✓
Difficult, correct	i was feeling pretty anxious all day but my first day at work was a very good day and that helped a lot	fear	✓ (1.00)	joy	✓
Difficult, correct	i do feel welcomed but it s a little weird	joy	✓ (1.00)	surprise	✓
Difficult, correct	i just feel extremely comfortable with the group of people that i dont even need to hide myself	joy	✓ (1.00)	anger	✓
Failure (G1)	i chant the invocation and feel his force supporting me as i teach	joy	love (0.63)	love	✓
Failure (G2)	when i learnt that my best friend had failed the exams	anger	sadness (0.88)	✓	sadness
Failure (G3)	i started to feel so overwhelmed	fear	surprise (0.70)	surprise	surprise
Failure (G5)	i feel accepted and loved by a community of derby girls that i helped to create	joy	love (0.90)	love	love

The transformer handles texts where the emotion word is contrasted or softened ("anxious … but … a very good day" is still labelled fear). The baseline follows the words with the largest weights ("good", "weird"). The failures involve close emotions, and in the last example all three models agree on a label (love) that is arguably as plausible as the gold label (joy).

15 Error Analysis

The analysis covers the 146 test errors of the selected model and compares them with the baseline (219 errors) and BERT (144 errors) on the same texts. The model predictions come from the single test evaluation; nothing was re-run.

15.1 Confusion Pairs

Table 15: Largest true → predicted confusions on test (ordered by DistilBERT). The last column is the share of the true class that DistilBERT sends to that label.

True	Predicted	DistilBERT	Baseline	BERT	% of true class
joy	love	42	58	20	6.04
anger	sadness	12	8	17	4.36
anger	fear	12	7	7	4.36
fear	surprise	11	19	26	4.91
sadness	anger	11	23	5	1.89
surprise	fear	10	2	0	15.15
joy	surprise	7	11	7	1.01
love	joy	6	8	28	3.77
sadness	fear	6	8	2	1.03
fear	sadness	6	8	5	2.68

The errors are concentrated: the three largest pairs hold 66 of the 146 errors (45.2%), and every listed pair links two close emotions; no pair links opposite emotions such as joy and anger. In absolute numbers joy → love is the largest error, but relative to class size the worst case is surprise → fear (15% of all surprise texts), which explains the low surprise F1. The *direction* of an error shows a model's bias: DistilBERT pushes joy towards love (42 vs. 6 the other way), while BERT pushes love towards joy (28 vs. 20) and over-predicts surprise (fear → surprise 26, never surprise → fear; surprise precision 0.65, recall 0.94).

15.2 Error Groups

The pairs are merged into groups that share a hypothesised cause; G1–G3 and "other" partition the errors, so each error is counted once.

Table 16: Error groups on test: number of errors (share of the model's errors), hypothesised cause and possible improvement.

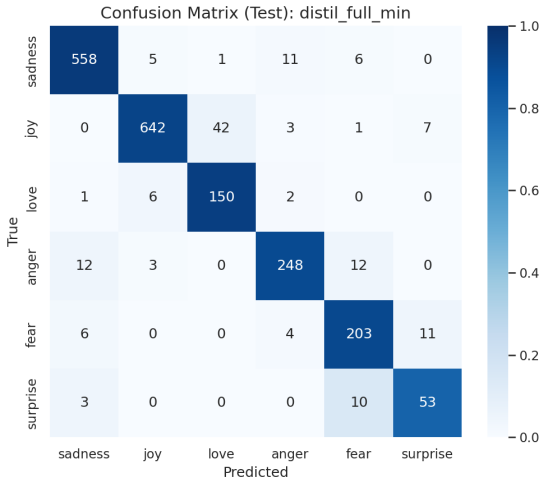
Group	DistilBERT	Baseline	BERT	Hypothesised cause	Possible improvement
G1 joy ↔ love	48 (32.9%)	66 (30.1%)	48 (33.3%)	Love texts often describe a happy feeling about someone, so the classes share words; class weights (love 2.07 vs. joy 0.50) push ambiguous texts to love	More love examples; per-class decision thresholds tuned on validation
G2 sadness / anger / fear	51 (34.9%)	61 (27.9%)	45 (31.2%)	One negative word (hurt, upset, stressed) fits several negative emotions; one text can hold more than one emotion	Multi-label annotation; use more context than the single emotion word
G3 fear ↔ surprise	21 (14.4%)	21 (9.6%)	26 (18.1%)	Surprise is the smallest class (564 training texts) and shares words such as "overwhelmed", "shocked" with fear	More surprise data or augmentation; a surprise threshold tuned on validation
Other pairs	26 (17.8%)	71 (32.4%)	25 (17.4%)	Scattered errors, mostly keyword-driven for the baseline	–

The transformers remove most of the baseline's scattered errors ("other" falls from 71 to 26) but not the three structural groups: G1 has the same size for both transformers, only with opposite directions, which suggests that the joy/love boundary is unclear in the labels themselves rather than a weakness of one model.

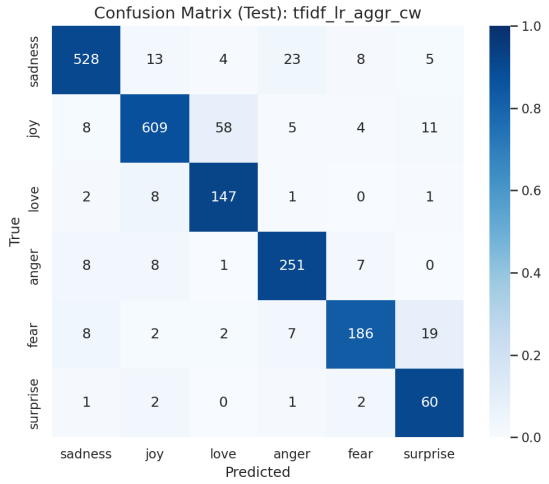
15.3 Label Noise and Text Properties

Possible label noise. On 50 test texts all three models give the same wrong label; they make up 34.2% of DistilBERT's errors. Three different models agreeing against the gold label points to a doubtful label or a truly ambiguous text. The 14 test texts whose copy in train or validation had another label (removed by filtering) support this: DistilBERT gets only 5 of them right, and on all 9 others it predicts exactly the label of the removed copy, e.g. "i feel like i am in paradise kissing those sweet lips make me feel like i dive into a magical world of love" is labelled joy in test and love in its training copy, and DistilBERT predicts love.

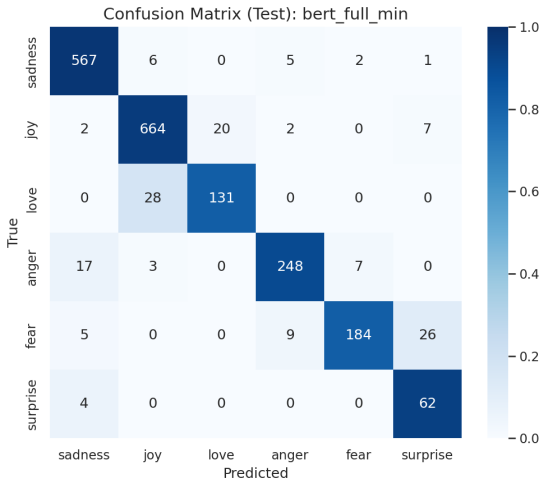
Where context helps. The advantage of DistilBERT over the baseline is largest on long texts (≥ 35 words: accuracy 0.920 vs. 0.806), texts with "but" (0.929 vs. 0.854) and texts with a negation word (0.912 vs. 0.857), and smallest on short texts (≤ 11 words: 0.943 vs. 0.924). The model is also overconfident when wrong: mean confidence 0.985 on correct and 0.796 on wrong predictions, with 47 of the 146 errors above 0.9.



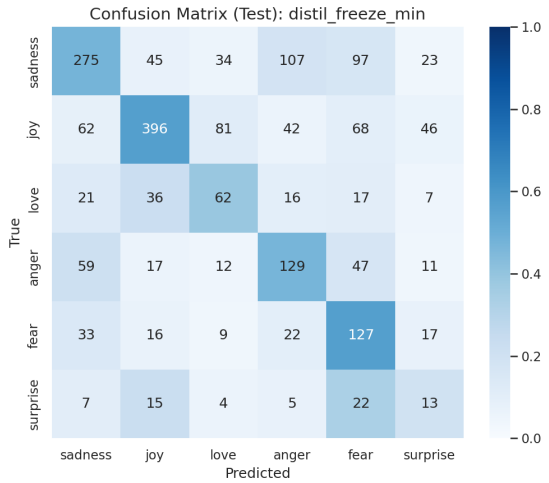
(a) DistilBERT full fine-tune (selected), test Macro-F1 0.8902



(b) TF-IDF + LR, aggressive text (selected baseline), 0.8548



(c) BERT full fine-tune, 0.8881



(d) DistilBERT frozen encoder, 0.4199

Figure 9: Test confusion matrices of the seed-42 main runs.

16 Comparison and Discussion

1. **Fine-tuning beats the baseline, by a clear but moderate margin.** DistilBERT exceeds the selected baseline by +0.0354 test Macro-F1 and +0.0365 accuracy, improves all six classes (most for love, +0.060 F1), and the difference is not chance: 124 texts are right only for DistilBERT, 51 only for the baseline (McNemar exact $p \approx 3 \times 10^{-8}$). The gain comes from texts where one emotion word is not enough (long, contrastive or negated texts); on short texts the baseline is almost as good, which agrees with the EDA finding that most labels follow a single emotion word.
2. **A bigger encoder does not help here.** BERT and DistilBERT are tied on validation and test and make the same number of errors in each group, while BERT costs about twice as much. On this dataset the bottleneck appears to be the labels and the close classes, not model capacity.
3. **The encoder must be adapted.** With a frozen encoder the model is worse than a bag of n-grams. The `[CLS]` vector of a model pre-trained with masked-language modelling does not linearly separate these emotions, and the recipe gives the head too few effective updates.

Freezing saves training time only, not inference time.

4. **Simple choices matter for the baseline.** Class weights (+0.108 test Macro-F1) and stopword removal with a negation keep-list (+0.018) together move the baseline from 0.7289 to 0.8548, closing much of the gap to the transformer at a tiny fraction of its cost.
5. **Trade-off.** If CPU latency or memory matters (1.3 ms vs. 60 ms per text, 1 MB vs. 256 MB) the improved baseline is a reasonable choice; if quality on harder texts matters, DistilBERT is the best quality per cost; BERT is not worth its extra cost here.
6. **What cannot be concluded.** The ranking of BERT and DistilBERT; whether a frozen encoder would work with a tuned learning rate or more epochs; whether the stopword gain holds for tuned TF-IDF settings; and the absolute scores without hyperparameter search. Three seeds give only a rough estimate of the spread.

17 Limitations and Conclusion

Limitations.

- **Data:** one English dataset of short, informal texts with machine-generated labels; conflicting duplicates show label noise, 14 test texts carry disputed labels, and near-duplicates (texts differing by a few words) were not audited. The test set is small for surprise (66 texts), so its F1 moves by about 0.015 per error.
- **Method:** no hyperparameter search for any model; the frozen-encoder result holds for one recipe; no per-class thresholds; single-label output although some texts express several emotions.
- **Experiments:** three seeds per transformer and a single run per baseline setting; timings come from a shared Colab machine (one CPU thread, one T4) and GPU memory was not measured.

Conclusion. *Research question: does fine-tuning a pretrained Transformer beat a classical TF-IDF baseline on six-class emotion classification, and at what cost?* Yes: fully fine-tuned DistilBERT reaches test Macro-F1 0.8902 (accuracy 0.9270) against 0.8548 for the best baseline, with gains on every class and mostly on texts that need context, at about 45 times the CPU latency and 650 times the parameters. BERT gives no measurable improvement for twice the cost, and freezing the encoder does not work under the same recipe. The remaining errors lie between close emotions and partly reflect doubtful labels.

Future work. Tune per-class thresholds on validation for love and surprise; clean or re-annotate the conflicting and all-models-wrong texts, or move to multi-label targets; tune the learning rate and the number of epochs of the frozen head; and shrink DistilBERT for CPU use (quantisation or further distillation).

18 Reproducibility

- **Code:** `notebooks/A2_emotion.ipynb` in the repository, run top to bottom on a Colab T4 ("Run all"); the full run from scratch takes about 45 minutes. The repository README gives installation, data preparation, training and evaluation steps.
- **Configuration:** one configuration cell holds every setting; the exported configuration file, hardware file and all result tables are in the `results/` folder of the repository.
- **Data and seeds:** dataset revision pinned; seeds 42, 43, 44; the training batch order depends only on the seed.

- **Checkpoints:** each run keeps its best-validation epoch on Google Drive; the whole Drive folder is shared (link on the title page): all 12 checkpoints in `A2/checkpoints/` (one subfolder per transformer run, one `.joblib` file per baseline) and the selected DistilBERT model with the selected baseline as one archive, `A2/checkpoints_final.zip` (257.3 MB). The repository README explains how to download a run and load it. A loading switch reuses saved checkpoints instead of retraining; turning it off rebuilds all 12 runs.
- **Experiment logs:** every reported number is traceable to a W&B run (Table 17); each run stores its configuration, per-epoch curves and, in its summary, the test scores.
- **Commit:** tag `a2-final`, the final commit of Assignment 02 on the `main` branch (submitted 11 Nov 2026).

Table 17: W&B run identifiers (project `goat-a2-emotion`).

Run	ID	Run	ID
`tfidf_lr_min_nocw`	7wlsxtx8	`bert_full_min`	ulss79nz
`tfidf_lr_min_cw`	pwfi016q	`bert_full_min_s43`	pil00scg
`tfidf_lr_aggr_cw`	3ojjzuey	`bert_full_min_s44`	noxjos1c
`distil_full_min`	lnuy747a	`distil_freeze_min`	7rugvdvg
`distil_full_min_s43`	psflj88c	`distil_freeze_min_s43`	83s806pr
`distil_full_min_s44`	llw3bte9	`distil_freeze_min_s44`	k1uomhhg

The public project holds these 12 runs plus one run that is not used anywhere (`x3xl0d1a`, one epoch, no test scores): the start of the final rerun, stopped and restarted. In each run summary, *val_macro_f1* is the value of the *last* epoch; the saved (best) epoch can differ (BERT seed 42 and frozen seed 43, Table 8), so selected scores are those of Table 8 and the test scores in the summaries.

19 AI Usage Disclosure

Following handbook §5, Table 18 records the one AI tool used in this assignment. The detailed log is `AI_USAGE.md` in the repository.

Table 18: AI usage disclosure (handbook §5.2 mandatory fields).

Field	Disclosure
1. Tool and model	Claude Code (Anthropic), model Claude Opus 5.5 (one session on Claude Sonnet 5.5, 25 Oct 2026)
2. Member who used it	Võ Lê Thành Triệu
3. Time / stage	25–28 Oct 2026 (M2: EDA, filtering, baseline, DistilBERT and BERT, preliminary results, report page) and 4–11 Nov 2026 (M3: controlled experiments, test evaluation, error analysis, compute cost, export, checkpoints section, Hugging Face model repository and live demo, this report, presentation slides and video script)
4. Purpose	experiment-design advice, coding assistance, debugging, result checking and analysis, concept explanation, web page design and report drafting (including the LaTeX source of this report), deployment of the selected models to Hugging Face (model repository and in-browser demo), slide generation and video-script drafting
5. Affected sections	the notebook (all blocks), the assignment page, README, Sections 1–17 of this report, the Hugging Face model repository and demo (`hf_space/`), the presentation slides and the video script
6. Representative prompts	"Write block 8: apply filtering rules 1–4 from proposal §5 … assert no overlap, $\geq$ 5,000 train rows and 6 classes per split." — "Write blocks 25 and 26: repeated inference timing, cost table and full export." — "Write the LaTeX report from the files in results/ only, following the handbook and our template; no code in the report." — "create slide for presentation for me include with crucial number and table for me, keep layout clean, include graph in results"
7. How the output was edited and verified	every cell was read, run on Colab and explained by the member before it was kept; key numbers (split sizes, class weights, Macro-F1, seed means, McNemar counts, error-group counts) were recomputed from the exported CSV files; every number in this report was traced to `results/` or the run notebook; the text was read and edited by the member; the slides were read again and every number on them checked against `results/` and this report, and the video script was read again and rewritten by the member in their own words; the models converted for the demo gave the same prediction as the original checkpoints on all 2,000 test texts
8. Responsible member	Võ Lê Thành Triệu
9. Verification sources	the course handbook; documentation of scikit-learn, PyTorch, Hugging Face transformers, datasets, Hub and transformers.js, W&B; the papers in the references; the Colab outputs
Not used for	creating data, metrics, examples or results; all decisions (filtering rules, class weights tested on validation, the experiments and the freeze threshold of 0.02, adding BERT and 3 seeds, model selection rules fixed before any test result, final baseline, short form of the error analysis) were made by Võ Lê Thành Triệu; the AI suggested options and made no decisions

19.1 Declaration

I, **Võ Lê Thành Triệu** (2453297), declare that: (1) I carried out Assignment 02 and take responsibility for all code, numbers, figures and text in this submission; (2) the AI tool above was used as an assistant under my review, and I can explain every part of the work; (3) no data, result, checkpoint, parameter count or reference was fabricated — every number comes from runs I executed on Google Colab; (4) the test set was used only after model selection.

Signature: *Võ Lê Thành Triệu* **Date:** 11 November 2026

References

[1] E. Saravia, H.-C. T. Liu, Y.-H. Huang, J. Wu, and Y.-S. Chen, "CARER: Contextualized affect representations for emotion recognition," in *Proc. EMNLP*, 2018, pp. 3687–3697. doi: 10.18653/v1/D18-1404. Dataset: https://huggingface.co/datasets/dair-ai/emotion

[2] G. Salton and C. Buckley, "Term-weighting approaches in automatic text retrieval," *Information Processing & Management*, vol. 24, no. 5, pp. 513–523, 1988. doi: 10.1016/0306-4573(88)90021-0

[3] A. Vaswani *et al.*, "Attention is all you need," in *Advances in NeurIPS*, 2017. https://arxiv.org/abs/1706.03762

[4] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of deep bidirectional transformers for language understanding," in *Proc. NAACL-HLT*, 2019, pp. 4171–4186. doi: 10.18653/v1/N19-1423

[5] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter," *arXiv:1910.01108*, 2019. https://arxiv.org/abs/1910.01108

[6] I. Loshchilov and F. Hutter, "Decoupled weight decay regularization," in *Proc. ICLR*, 2019. https://arxiv.org/abs/1711.05101

[7] T. Wolf *et al.*, "Transformers: State-of-the-art natural language processing," in *Proc. EMNLP: System Demonstrations*, 2020, pp. 38–45. doi: 10.18653/v1/2020.emnlp-demos.6

[8] A. Paszke *et al.*, "PyTorch: An imperative style, high-performance deep learning library," in *Advances in NeurIPS*, 2019. https://arxiv.org/abs/1912.01703

[9] F. Pedregosa *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011. https://jmlr.org/papers/v12/pedregosa11a.html

[10] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947. doi: 10.1007/BF02295996

[11] L. Biewald, "Experiment tracking with Weights and Biases," 2020. Software available from https://wandb.ai